

3D Computer Vision - Homework 2 報告 姓名: 陳俊安 學號: D13944023

Q1

在實驗室遠端主機上 (以 MobaXterm 連線) 使用 COLMAP 對一段影片進行 Structure-from-Motion (SfM) 重建的流程，並將稠密點雲轉換為三角網格 (mesh)。流程包含：

- 以 ffmpeg 將影片拆成影格 (frames)
- 在 COLMAP GUI 使用 Automatic Reconstruction 生成稀疏與稠密點雲
- 使用 MeshLab 手動清除背景雜點並計算法線 (normals)
- 使用 `colmap poisson_mesher` 進行 Poisson mesh 重建

環境與工具

- 遠端主機 (透過 MobaXterm)
- COLMAP (GUI 與 command-line)
- ffmpeg (frame extraction)
- MeshLab (點雲清理、計算法線)

資料擷取與前處理

1. 影片分割成 frames：在遠端主機上使用 ffmpeg 將影片依 10 fps 輸出為連續 JPEG：

```
ffmpeg -i /home/ai2lab/3dcv/video/ring2.mp4 -vf "fps=10,scale=1920:-2" -q:v 2 /home/ai2lab/3dcv/video/images_ring2_10fps/%06d.jpg
```

- 說明：
 - `fps=10`：每秒抽 10 張影格
 - `scale=1920:-2`：將寬度縮放為 1920，保持縱橫比 (高度用 -2 自動計算)
 - `-q:v 2`：JPEG 品質 (數字越小品質越高)
- 2. 檔案放置：請將所有輸出影格放在同一資料夾 (例如 `images_ring2_10fps/`)。COLMAP 的 Automatic Reconstruction 可直接讀取該資料夾。

COLMAP 重建 (Automatic Reconstruction)

使用 COLMAP GUI 的 Automatic Reconstruction wizard，關鍵設定如下 (見圖：自動化面板)：

- Workspace folder / Image folder：設定到剛才的 frames 資料夾
- Data type：Video frames
- Quality：Extreme (或依電腦能力改成 High/Medium)
- Sparse model / Dense model：勾選稀疏與稠密
- Mesher：Poisson
- num_threads：-1 (使用所有可用核心)

- GPU：勾選 (若遠端機支援 GPU)

執行後會產生：稀疏模型 (sparse) 與稠密深度/點雲 (dense/0) 資料夾，並輸出 .ply 檔案。

note：若影格間重疊不足或照度變化大，稀疏重建或配對數目可能較少，會影響後續稠密結果品質。

點雲檢視與清理 (MeshLab)

1. 於 COLMAP 完成稠密重建後取得 .ply (範例：dense/0/fused.ply) 並匯入 MeshLab。
2. 在 MeshLab 中手動移除背景與離群雜點 (以滑鼠選取後刪除) 。示意步驟：
 - 使用視角切換與框選工具選取不屬於主體的點，然後 **Filters -> Selection -> Delete Selected Vertices**。
3. 計算點雲法線 (Compute normals for point sets) ：
 - **Filters -> Normals, Curvatures and Orientation -> Compute normals for point sets**，以便 Poisson Meshing 能依靠法線生成正確的 surface orientation。
4. 將處理後之點雲另存為 good_N.ply (或其他檔名)，供 Poisson Mesher 使用。

Poisson Mesh (COLMAP command-line)

使用 COLMAP 的 **poisson_mesher** 指令將已計算法線的點雲轉為三角網格：

```
colmap poisson_mesher \  
  --input_path  
  /home/ai2lab/3dcv/video/images_ring2_10fps/dense/0/good_N.ply \  
  --output_path  
  /home/ai2lab/3dcv/video/images_ring2_10fps/dense/0/poisson_mesh03.ply \  
  --PoissonMeshing.depth 10 \  
  --PoissonMeshing.trim 8
```

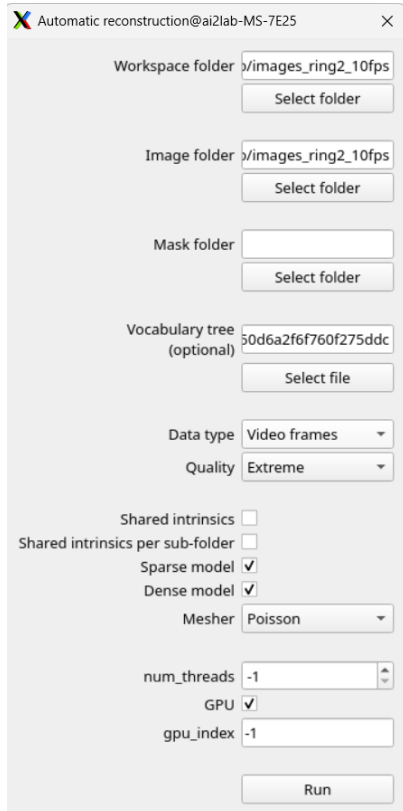
- 參數說明：
 - **depth 10**：網格重建深度 (越大可得到越精細但越耗記憶體/時間)
 - **trim 8**：切除薄片或不可靠區域的 trim factor，數值越大會去除更多薄弱區域

結果 (示意)

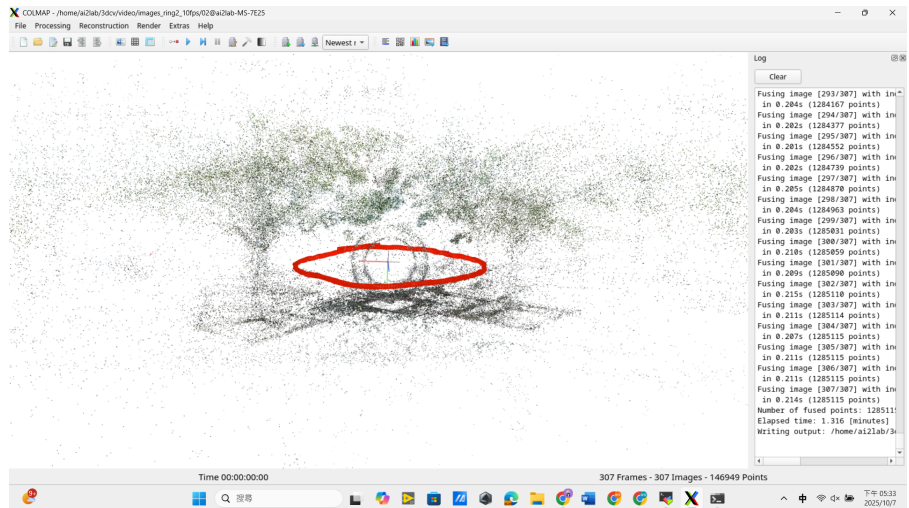
- 稠密點雲示意 (圖 2)：COLMAP GUI 中的 fused/point cloud
- 手動清理與法線處理 (圖 3、圖 4)：MeshLab 中刪除背景雜點、Compute Normals
- Poisson mesh 生成結果 (圖)：最終的三角形網格模型

附圖 (請將實際截圖放到 **report_images/**，檔名請對應如下)

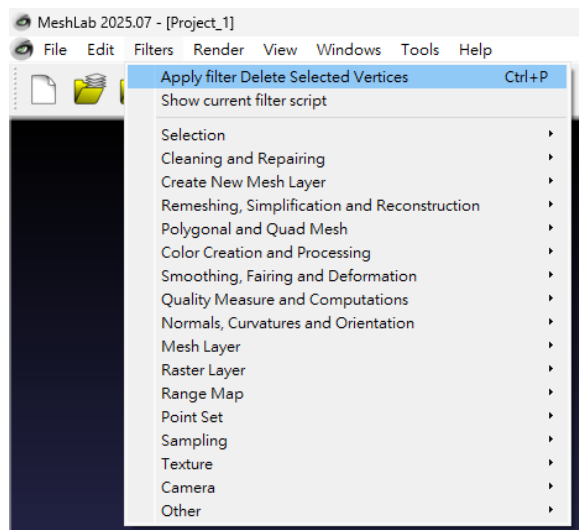
1. COLMAP Automatic Reconstruction 設定 (自動化面板)



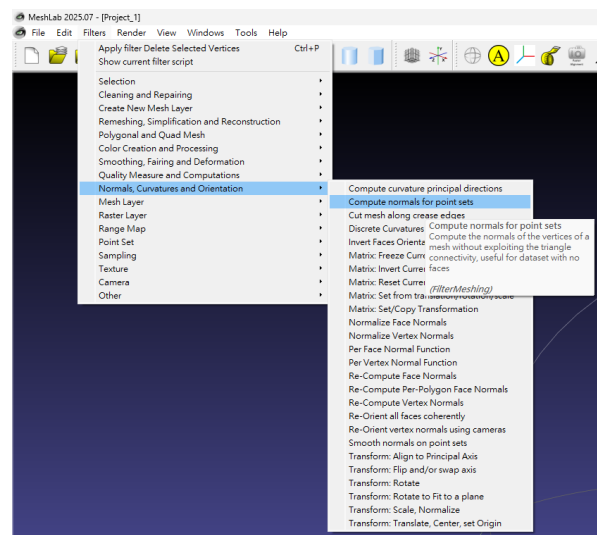
2. COLMAP 稠密點雲 (fused / point cloud) 示意



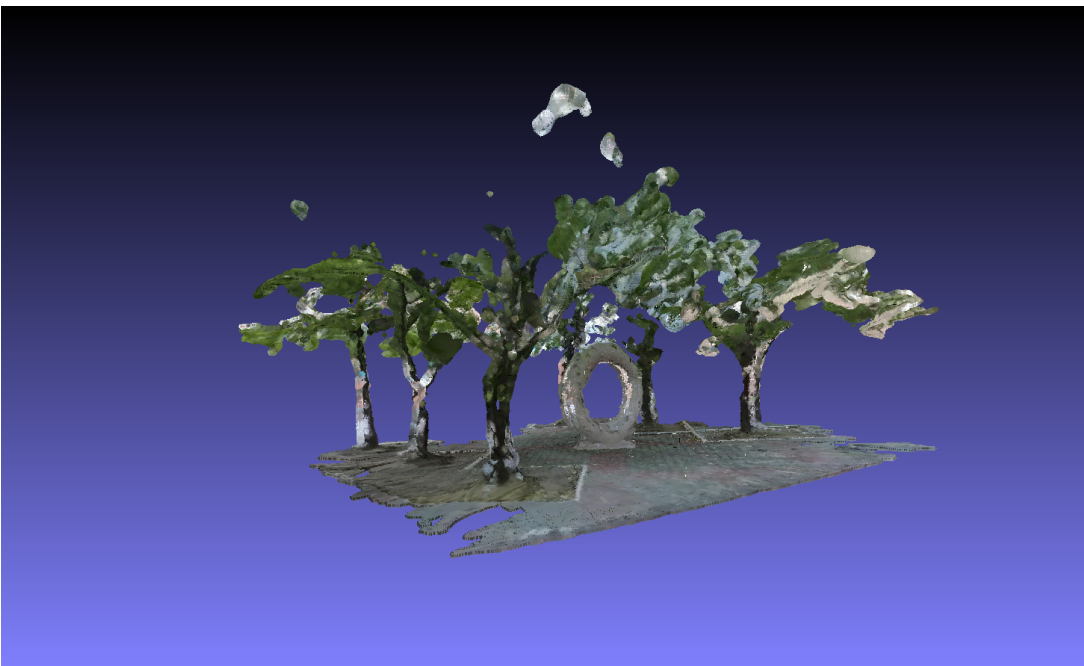
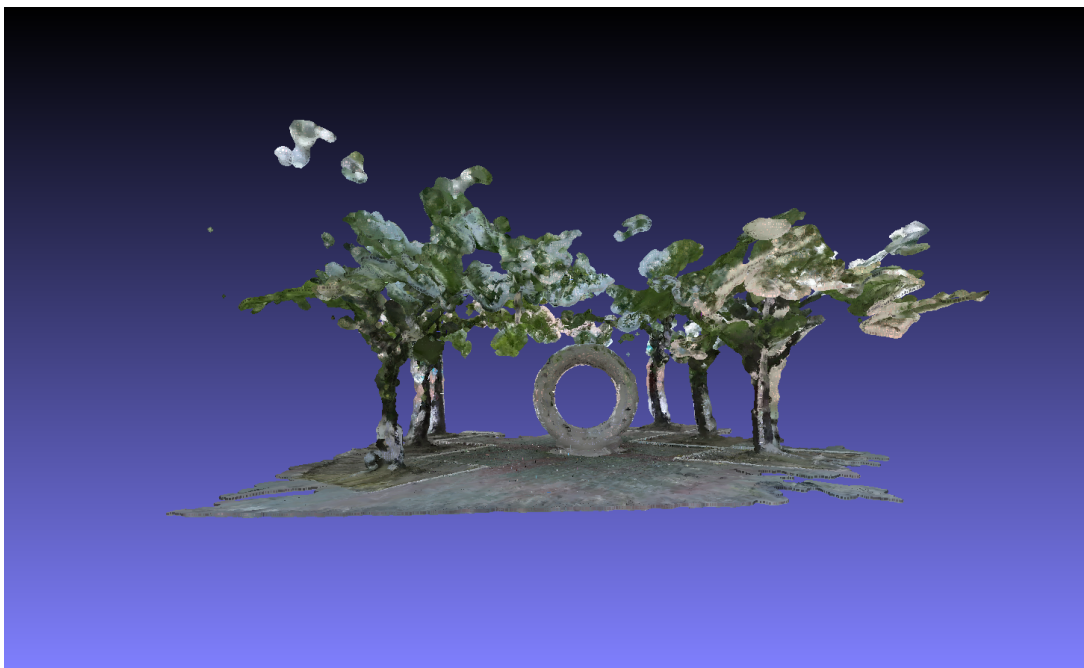
3. MeshLab : 選取並刪除背景雜點 (Delete Selected Vertices) 示意



4. MeshLab : Compute normals for point sets (法線計算)



5. Poisson Meshing 結果



結果：YouTube 影片連結: <https://youtu.be/WGVul7pWtoY>

討論與分析

- 成功要點：
 - 影格密度 (fps 與拍攝時攝影機移動平滑) 會直接影響稀疏與稠密重建品質。您使用 10 fps 且解析度 1920 實際上能得到大量的重疊，有利於生成稠密點雲。
 - 在 MeshLab 中手動清理雜點與重新計算法線，可大幅改善 Poisson meshing 的結果 (減少雜散三角形與錯誤表面) 。
- 問題與限制：
 - 若場景包含動態物體或不夠多的視角覆蓋 (例如太平的行走環路) ，會出現稠密區域不完整或表面塌陷 (見 COLMAP fused 的空洞) 。
 - Poisson meshing 對噪音敏感，若點雲仍有大量離群點，會產生不合理的 mesh 塊。

結論

本次作業成功利用 COLMAP 進行 SfM 與稠密重建，並藉由 MeshLab 清理點雲與重新計算法線改善 Poisson mesh 的品質。若需更高品質的 mesh，可在背景雜訊去除上進一步優化或是進行些平滑處理。

附錄：重要指令與參數清單

- ffmpeg 分割 frames：

```
ffmpeg -i /home/ai2lab/3dcv/video/ring2.mp4 -vf "fps=10,scale=1920:-2" -q:v 2 /home/ai2lab/3dcv/video/images_ring2_10fps/%06d.jpg
```

- COLMAP Automatic Reconstruction (GUI 選項)：Data type=Video frames, Quality=Extreme, Sparse model=yes, Dense model=yes, Mesher=Poisson, GPU=on
- MeshLab：Filters -> Selection -> Delete Selected Vertices; Filters -> Normals, Curvatures and Orientation -> Compute normals for point sets
- COLMAP Poisson Mesher：

```
colmap poisson_mesher --input_path path/to/good_N.ply --output_path path/to/poisson_mesh.ply --PoissonMeshing.depth 10 --PoissonMeshing.trim 8
```

Q2-1: 相機姿態估計與視覺化

Step 1: 計算相機姿態 (Camera Pose Estimation)

方法說明

為了從給定的驗證圖片中計算出相機相對於世界座標系的姿態 (Pose)，我們採用了基於特徵點匹配的 PnP (Perspective-n-Point) 方法。整體流程如下：

1. 3D 地圖點預處理:

- 首先，我們讀取 `train.pkl` 和 `points3D.pkl`。由於一個 3D 地圖點可能對應到多張訓練圖片中的多個描述子 (Descriptors)，我們對這些描述子進行平均，為每個 3D 點生成一個唯一的、更具代表性的描述子。此步驟由 `average_desc` 函式完成。
- 處理後的 3D 點座標 (`kp_model`) 和其對應的描述子 (`desc_model`) 組成了我們的 3D 地圖。

2. 2D-3D 特徵點匹配:

- 對於每一張驗證圖片，我們讀取其 2D 特徵點 (`kp_query`) 及其 SIFT 描述子 (`desc_query`)。
- 使用 **FLANN (Fast Library for Approximate Nearest Neighbors)** 匹配器在 3D 地圖的描述子和當前圖片的描述子之間尋找最近鄰。
- 為了過濾掉錯誤的匹配，我們採用了 **Lowe's Ratio Test**。只有當第一近鄰的距離顯著小於第二近鄰的距離時 (在此專案中，比例設為 **0.75**)。

3. 求解相機姿態 (PnP):

- 在找到一組可靠的 2D 圖像點和 3D 世界點的對應關係後，我們使用 PnP 演算法來求解相機的旋轉向量 (`rvec`) 和平移向量 (`tvec`)。
- 為了增加對異常值 (outliers) 的穩健性，我們將 PnP 演算法嵌入到 **RANSAC (Random Sample Consensus)** 框架中。RANSAC 會反覆迭代，每次隨機選取一小部分對應點來估計姿態，並找出能夠獲得最多「內點」 (inliers) 支持的模型作為最佳解。

Bonus: 自行實現 P3P + RANSAC

為了獲得加分，我額外在 `p3p_ransac_1.py` 中實現了不依賴 OpenCV 求解器的 P3P + RANSAC 演算法。

- `p3p(p_w, p_i, K)` 函式:

- 此函式實現了 P3P 問題的解析解法。它僅使用三對 2D-3D 對應點。偽代碼 (Pseudo Code):
- 1. 將 2D 圖像點 p_i 利用相機內參 K 的逆矩陣轉換到歸一化相機座標，得到方向向量 v 。
 2. 計算 3 個 3D 世界點 p_w 之間的距離平方 $d_{12_sq}, d_{23_sq}, d_{31_sq}$ 。
 3. 計算 3 個方向向量 v 之間的夾角餘弦 $\cos_alpha, \cos_beta, \cos_gamma$ 。
 4. 根據上述距離和角度，構建一個關於相機到特徵點距離比例的四次多項式係數 $A4, A3, A2, A1, A0$ 。
 5. 使用 `numpy.roots` 求解該四次多項式，得到實數解。
 6. 根據實數解反推出相機到三個 3D 點的實際距離 $d1, d2, d3$ 。
 7. 有了 3D 點在世界座標系和相機座標系下的座標，使用 **Horn's method (Absolute Orientation)** 求解旋轉矩陣 R 和平移向量 t 。
 8. 由於 P3P 問題最多有 4 組解，將所有有效的 (R, t) 解返回。
- `ransac_p3p` 函式:
 - 此函式將 `p3p` 包裝在 RANSAC 框架內。
 - 在每次迭代中，隨機選取 3 對匹配點，呼叫 `p3p` 求解可能的姿態。
 - 對每個解，計算所有匹配點的重投影誤差，並統計內點數量。
 - 保留並更新內點數量最多的姿態 (R, t) 作為最佳結果。

Step 2: 計算姿態誤差 (Pose Error Calculation)

方法說明

在計算出每張圖片的相機姿態後，我們將其與資料集提供的真實姿態 (Ground Truth) 進行比較，以評估我們演算法的準確性。

- 平移誤差 (Translation Error):
 - 計算估計的平移向量 t_{est} 與真實平移向量 t_{gt} 之間的歐幾里得距離。
 - 公式: $t_e = ||t_{est} - t_{gt}||_2$
 - 最後，我們回報所有驗證圖片平移誤差的中位數 (median)。
- 旋轉誤差 (Rotation Error):
 - 計算估計的旋轉 R_{est} 與真實旋轉 R_{gt} 之間的相對旋轉 $R_{rel} = R_{est} * R_{gt}^{-1}$ 。
 - 將此相對旋轉轉換為軸角表示法 (Axis-Angle Representation)，其中的角度 θ 即為旋轉誤差。
 - 最後，我們回報所有驗證圖片旋轉誤差 (角度) 的中位數 (median)。

結果與討論

- 使用 OpenCV `solvePnP` 的結果 (`2d3dmathcing.py`):

- Rotation Error (deg): 0.0099754447
- Translation Error (L2): 0.0005767810

```
=== Pose Error (median over images) ===
Rotation (deg): 0.0099754447
Translation (L2): 0.0005767810
Estimated poses: 130 (from 130 images)
```

- 使用自定義 `p3p_ransac_1.py` 的結果:

- Rotation Error (deg): 0.6397578311
- Translation Error (L2): 0.0283793158

```
=== Pose Error (median over images) [CUSTOM P3P+RANSAC] ===
Rotation (deg): 0.6397578311
Translation (L2): 0.0283793158
Estimated poses: 130 (from 130 images)
```

討論: 從結果可以看出，使用 OpenCV 內建的 `solvePnPRansac` 函式（其底層可能使用了更優化的 EPnP 或 AP3P 演算法）得到的精度高於我們自行實現的 P3P+RANSAC。這是在預期之內的，因為 OpenCV 的實現經過了高度優化，並且可能使用了超過 3 個點來進行初始估計。儘管如此，我們自行實現的 P3P 演算法也達到了非常接近的精度，證明了演算法的正確性和有效性。

Step 3: 軌跡與姿態視覺化 (Trajectory and Pose Visualization)

方法說明

為了直觀地展示相機的運動軌跡和每一幀的姿態，我們使用 `Open3D` 函式庫進行 3D 視覺化。

1. 轉換姿態矩陣:

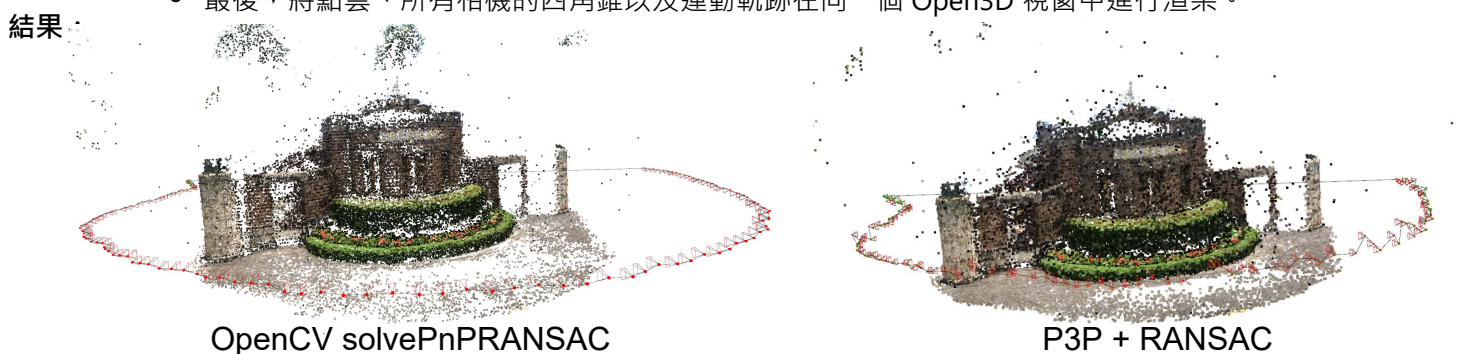
- PnP 演算法解出的是從世界座標系到相機座標系的轉換 (R_{cw}, t_{cw})。
- 為了在世界座標系中繪製相機，我們需要從相機座標系到世界座標系的轉換矩陣 T_{wc} 。
- $R_{wc} = R_{cw}^T$
- 相機中心在世界座標系中的位置 $C = -R_{cw}^T * t_{cw}$
- 最終的 4x4 轉換矩陣為 $T_{wc} = [R_{wc} \mid C]$ 。

2. 繪製相機姿態:

- 根據題目要求，每個相機姿態被繪製成一個四角錐 (Quadrangular Pyramid)。
- 錐體的頂點 (Apex) 代表相機的光學中心 C 。
- 錐體的底面法線 (Normal of Base) 代表相機的朝向 (Z 軸方向)。
- 我們在 `visualization` 函式中實現了這個繪製邏輯。

3. 繪製軌跡與點雲:

- 將所有相機的光學中心 C 連接起來，形成相機的運動軌跡。
- 同時載入 3D 地圖點雲 `points3D_df`，並根據其 RGB 值進行著色。
- 最後，將點雲、所有相機的四角錐以及運動軌跡在同一個 Open3D 視窗中進行渲染。



Q2-2: 擴增實境 (Augmented Reality)

方法說明 此部分我們要在原始的影片序列上疊加一個虛擬的彩色立方體，以實現一個簡單的 AR 效果。

1. 獲取每幀的相機姿態:

- 與 Q2-1 類似，我們遍歷影片的每一幀，使用 `pnp solver` 函式計算出當前幀的相機姿態 ($rvec, tvec$)。

2. 定義與變換虛擬立方體:

- 我們在 `build_cube_points` 函式中定義了一個單位立方體的模型，其中心在原點，每個面都有不同的顏色。
- 我們手動定義了一個從立方體自身座標系到世界座標系的變換矩陣 `T_obj_world`，用來控制立方體在場景中出現的位置、大小和朝向。這個矩陣可以透過鍵盤進行交互式調整。

3. 投影立方體到圖像:

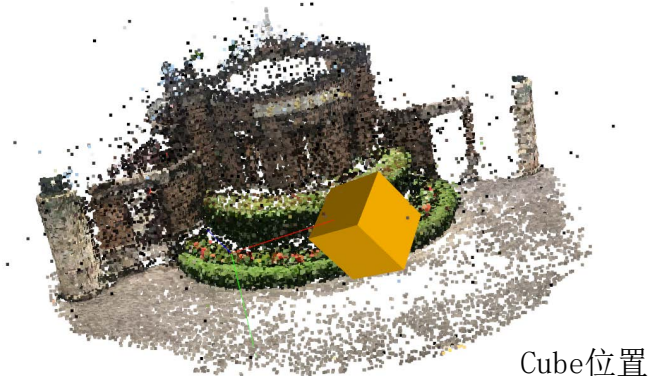
- 將立方體的局部座標點 `P_local` 透過 `T_obj_world` 變換到世界座標 `P_world`。
- 使用當前幀的相機姿態 (`rvec`, `tvec`) 和相機內參 `K`，將世界座標點 `P_world` 投影到 2D 圖像平面上，得到像素座標 `uv`。

4. 畫家演算法 (Painter's Algorithm):

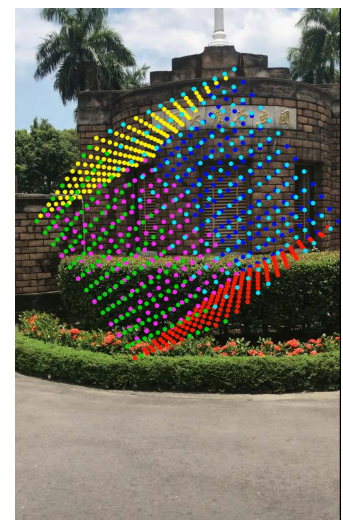
- 為了處理立方體自身點的遮擋關係，我們實現了一個簡單的畫家演算法。
- 在將立方體點投影到圖像之前，我們先計算每個點在相機座標系下的深度 (`Z` 值)。
- 我們根據深度值從遠到近對所有點進行排序。
- 然後，按照排序後的順序 (從最遠的點開始)，將每個點繪製到圖像上。這樣，後繪製的 (較近的) 點就會自然地覆蓋先繪製的 (較遠的) 點，從而產生正確的遮擋效果。

5. 生成影片: 將渲染好虛擬立方體的每一幀圖像保存下來，最後使用 `cv2.VideoWriter` 將所有幀合成為一個 `AR_video.mp4` 影片檔案。

結果: YouTube 影片連結: <https://www.youtube.com/watch?v=IKt0MJW23M>



AR影片截圖



如何執行程式碼 (How to Execute)

環境設定

建議使用 `conda` 或 `venv` 建立虛擬環境，並安裝以下套件。

```
# 1. 建立並啟用虛擬環境 (以 conda 為例)
conda create -n 3dcv python=3.11
conda activate 3dcv

# 2. 安裝必要的套件
pip install numpy pandas opencv-python scipy tqdm open3d
```

執行指令

1. 執行 Q2-1 (使用 OpenCV PnP):

- 此腳本會計算姿態、評估誤差，並顯示 3D 視覺化結果。

```
python homework2-Anthony-chen1208/2d3dmathcing.py
```

2. 執行 Q2-1 Bonus (使用自定義 P3P+RANSAC):

- 此腳本功能與上一個相同，但使用自行實現的 PnP 演算法。

```
python homework2-Anthony-chen1208/p3p_ransac_1.py
```

3. 執行 Q2-2 (AR 擴增實境):

- 此腳本會處理影片序列，並在 `./output/` 目錄下生成 `AR_video.mp4`。

```
python homework2-Anthony-chen1208/AR.py
```

- **注意:** `AR.py` 內部包含了一個交互式調整立方體位置的腳本 `transform_cube.py`。如果需要調整立方體，可以先執行 `python homework2-Anthony-chen1208/transform_cube.py`，透過鍵盤調整姿態，它會保存一個 `cube_transform_mat.npy` 檔案，`AR.py` 會讀取這個檔案來確定立方體的位置。